SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL 3)

Assessment Tool Weightage: 40%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

ANALYTICAL PROBLEM SOLVING

Q1. Apply your understanding of compiler design to demonstrate how a source program is processed through all phases of a compiler.

Consider the statement:

w=x y z+20

where w.x.y.z are of type real.

Illustrate the transformation of the above statement through each phase of the compiler, including:

(i). Lexical Analysis – Identify tokens and construct the symbol table.

(ii). Syntax Analysis – Show how the statement is validated using grammar rules.

(iii). Semantic Analysis – Verify type consistency and correctness of the expression.

(iv). Intermediate Code Generation – Represent the statement in an intermediate form.

(v). Code Optimization – Apply possible optimizations to improve efficiency.

(vi). Target Code Generation – Generate the final machine-level or assembly-like code.

Clearly show how the internal representation of the program changes at each phase and explain the role of each phase in the translation process. (10 Marks)

Q2. Construct appropriate regular expressions for the following subsets of {0,1}*:

(i) All strings beginning and ending with 1.

(ii) All strings containing the substring 01.

(iii) All strings with exactly one 0.

(iv) All strings with at least two 1's.

(v) All strings that do not end with 0. (10 Marks)

Q3. Apply your understanding of Formal Languages and Automata Theory to interpret regular expressions and construct the corresponding languages.

For each of the following regular expressions, construct and describe the language generated. Represent your answer by listing example strings and explaining the pattern of the language.

(i) (c+aa)

Construct the language and describe the possible forms of strings generated.

(ii) (a +b) abb

Determine the language and explain the structure of strings ending with a specific pattern.

(iii) (a b) aba

Construct the language and identify the mandatory substring condition.

(iii) (a+ab)

Generate the language and describe how repetition affects string formation.

(iv) (aba) +(a b)

Construct the combined language and explain the contribution of each part of the expression.

(10 Marks)

Q4. Apply your understanding of Compiler Design to design components of a lexical analyzer using Lex for a simple programming language.

Identifier Recognition using Lex

You are given a programming language with the following token specifications:

Keywords: if, else, while

Identifiers: A sequence of letters (uppercase and lowercase) and digits, starting with a letter

Constants: Integer values

Operators: +, -, *, /

Delimiters: (,), {, }, ;

(i) Construct a Lex regular expression that recognizes identifiers in this language.

(ii) Justify how your regular expression correctly identifies valid identifiers while avoiding conflicts with keywords.expressions distinguish between valid and invalid numeric formats with suitable examples.

(10 Marks)

Q5. Apply algebraic properties of regular expressions to prove the following identities:

(i) (0 1)* =(0+1)

(ii) (0+1) (0+1) =(0+1)

(iii) (0) =0

(iv) (0+1) 0(0+1) +(0+1) 1(0+1) =(0+1)*

(v) ε+0(0) =0 (10 Marks)

21/07/26.

COI - ATI - Compiler Design.

Analytical problem solving

Qn1. $W = X * Y - Z + 20$

analyze lexical, syntax, semantic, Intermediate code generat code optimization, target code generation - Explain each slot.

Source program,

$W = X * Y - Z + 20$

- W, X, Y, Z : real
- 20: integer constant

i) Lexical Analysis.

Role: Lexical analysis scans the source program character by character and converts it into the tokens. It also creates the symbol table.

Token Table.

Lexeme	Token
W	Identifier (ID)
=	Assignment operator
X	(ID)
*	Multiplication operator
Y	(ID)
-	Substration operator
Z	(ID)
+	Addition operator
20	Number (Num)

(ID, x)

(*)

(ID, y)

(-)

(ID, z)

(+)

(NUM, 20)

Symbol table

Identifier	Type	Address
w	Real	100
x	Real	104
y	Real	108
z	Real	112

Token Representation

$\langle id_1, w \rangle \langle = \rangle \langle id_2, x \rangle \langle * \rangle \langle id_3, y \rangle \langle - \rangle \langle id_4, z \rangle$

$\langle + \rangle \langle num, 20 \rangle$

ii) Syntax Analysis

Role: Syntax Analysis check whether the tokens follow the grammar ruler, of the programming language and constructs the parse tree.

Grammar:

$S \rightarrow id + E$

$E \rightarrow E + T \mid E - T$

$E \rightarrow T$

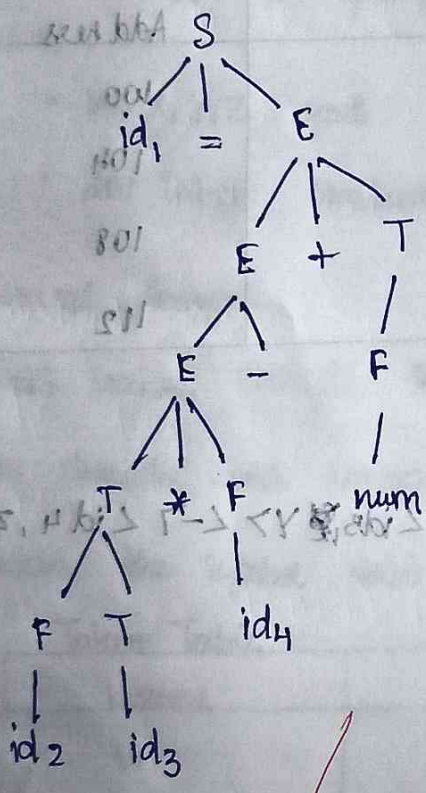
$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow id$

$F \rightarrow num$

Parse Tree,



Where,

$id_1 = W$

$id_2 = X$

$id_3 = Y$

$id_4 = Z$

iii) Semantic Analysis

Role: Checks the meaning of the problem by verifying declaration type compatability and valid operations.

Type checking

id1 (w) → Real

id2 (x) → Real

id3 (y) → Real

id4 (z) → Real

20 → Integer

Expression checking

id2 * id3 → Real

Real − Real → Real

Real + Integer

(20 → connected to 20.0 Real)

result → Real

id1 = Real

iv) Intermediate code generation,

Role: Converts validated sourse program into machine independent intermediate representation. (three Address code)

Three Address code

$T_1 = id_3 * id_3$

$T_2 = T_1 - id_4$

$T_3 = T_2 + 20.0$

$id_3 = T_3$

v) Code optimization

Role: improves the intermediate code by reducing unnecessary instructions and temporary variables. makes execution faster & more efficient.

$T_1 = id_2 * id_3$

$T_2 = T_1 - id_4$

$id_3 = T_2 + 20.0$

vi) Target code generation,

Role, Converts the optimized intermediate code into machine - level of assembly - level like instructions executed by the processor

Assembly - like code

```
MOV  R1, id2
MUL  R1, id3
SUB  R1, id4
APP  R1, #20.0
MOV  id1, R1
```

Q2. Construct Regular Expression $\{0,1\}^*$.

i) All strings begins and ends with 1.

RE, EX

$$1(0+1)^*1 + 1.$$

Explain,

* Starts with 1
* Ends with 1
* $(0+1)^*$ → allows combination of 0's and 1's in between.
* +1 includes the single-character string "1".

Ex: 1, 11, 101, 1101, 1000

ii) All strings contains the substring 01.

Regular Expression

$$(0+1)^* 01 (0+1)^*$$

* $(0+1)^*$ → allows any symbol before 01
* 01 → req. substring
* $(0+1)^*$ → allows any symbol before 01.

Ex: 01, 101, 010, 1011.

iii) All strings with exactly one 0.

RE

$$1^* 01^*$$

* 1^* → Any no of 1's before 0
* 0 → Exactly one 0
* 1^* → Any no of 1's after 0.

iv) All strings with at least two 1's.

RE,

$$(0+1)^* 1 (0+1)^* 1 (0+1)^*$$

* The expression guarentees at last two occurences of 1
* Any combination of 0's and 1's can appear before, between and after them

Ex: 11, 101, 1100, 1001.

v) All strings that do not end with 0.

RE, $(0+1)^*1) + \varepsilon$

* $(0+1)^*1$ → generate all strings ending with 1

* ε (epsilon) represents empty string which also does not end with 0.

Ex: ξ 1, 11, 101, 1101.

Qs. Construct Language generated by following RE

i) $(\varepsilon + aa^*)^*$

Language,

Sin u, $\boxed{aa^* = a^+}$ $\Rightarrow (a^*)^n = a^*$

the expression becomes.

$(\varepsilon + a^+)^*$ (generates all strings consisting only of 'a', including empty string).

Language:

$L = \{ \varepsilon, a, aa, aaa, aaaa \}$

Example Strings:

$\varepsilon, a, aa, aaa, aaaa, aaaaa$

Pattern:

* contains only the symbol 'a'.

* the empty string is also allowed

* Any number of a's can occur.

ii) $(a^* + b^*)^* abb$.

$(a^* + b^*)^*$ → generate any string over $\{a, b\}$.

So,

$(a^* + b^*)^* abb = (a+b)^* abb$.

Language,

$L = \{abb, aabb, babb, aaabb, ababbb\}$ pattern,

The L' contains any combination of a and b, but every string ends with "abb".

iii) $(a^* b)^* aba$

$(a^* b^*)^* aba = (a+b)^* aba$.

$L = \{aba, aaba, baba, ababa, baaba\}$ pattern

The 'L' contains any combination of a and b, every string ends with 'aba'.

iv) $(a+ab)^*$

→ Repeation chooses a either ab

Language → $L = \{\varepsilon, a, ab, aa, aab, aba, aba, aaa \ldots\}$

pattern,

The 'L' formed by repeating either "a" or "ab" any no. of times.

v) $(aba)^* + (a^* b^*)^*$

$(a^* b^*) = (a+b)^*$

$\Rightarrow (aba)^* + (a+b)^*$

$\Rightarrow (aba)^* + (a+b)^* \Rightarrow (a+b)^*$

$L = \{ \varepsilon, a, b, aa, bb, ab, ba, aba, bab, aabb, abab \}$

Pattern

All possible strings over $\{a,b\}$ including ε.

Qn4 Design components of a Lexical Analyzer using Lex -

i) Identifier (RE).

$$[A-Z \cdot a-z][A-Z \; a-z \; 0-9]^*$$

ii) Justification

* The 1st character must be a letter (A-Z) or (a-z)
* The remaings are letters or digit.
* The (*) operator allows zero or more letters or digits after first letter.

Valid	Invalid.
count	1count
X	@name
sum 1	abc $
Student 25	ZX
ABC 123	

Ans Prove following Identities, algebric properties of Regular Expressions.

i) $(0^*1^*)^* = (0+1)^*$

$(0^*1^*)^* = (0+1)^*$

property = closure property

$(R^*S^*)^* = (R+S)^*$

ii) $(0+1)^*\ (0+1)^* = (0+1)^*$

LHS. $(0+1)^*\ (0+1)^*$

Idempotent law

$R^*\ R^* = R^*$

iii) $(0^*)^* = 0^*$

LHS

$(0^*)^*$ => Star Law

$(R^*)^* = R^*$

$\Rightarrow 0^*$

iv) $(0+1)^* 0 (0+1)^* + (0+1)^* + 1 (0+1)^* = (0+1)^*$

~~LHS~~

$(0+1)^* 0 (0+1)^* + (0+1)^* + (0+1)^* = (0+1)^* (0+1)^*$

Property : Distributive Law.

$= (0+1)^* \ (0+1)^* \longrightarrow$ Positive closure.

$R^* = R^* \ R^*$

$R^* \ R^* = R^*$

Therefore $\Rightarrow (0+1)^*$

∴ Hence,

$(0+1)^* 0 (0+1)^* + (0+1)^* + 1 (0+1)^*$

$= (0+1)^*$

v) $\varepsilon + 0(0)^* = 0^*$

$\varepsilon + 0(0)^* = 0^* \Rightarrow$ Kleen star identity.

$R^* = \varepsilon + RR^* \quad (R=0)$

Hence Proved

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation